

Mastering Java CompletableFuture

Part 2: Composition

The critical difference between `thenApply` and `thenCompose`, parallel execution patterns, and why you should never use `ForkJoinPool.commonPool()` in production.



Morty

Follow

5 min read · Feb 4, 2026

The Power of Composition

In [Part 1](#), we learned the basics. But the real magic of `CompletableFuture` isn't just "run something async"—it's **composing** multiple async operations elegantly.

Think about a typical dashboard load:

- Fetch user data (100ms)
- Fetch recent orders (150ms)
- Fetch payment history (120ms)

Sequential approach: $100 + 150 + 120 = 370\text{ms}$ **Parallel approach:** $\max(100, 150, 120) = 150\text{ms}$

That's 2.5x faster. Same data, same operations, just smarter orchestration.

Let's learn how.

The Critical Difference: thenApply vs thenCompose

This is the #1 source of confusion (and bugs) with `CompletableFuture`. Get this right, and everything else clicks.

thenApply — Synchronous transformation

Use when your transformation **doesn't** involve another async call:

```
CompletableFuture<String> emailFuture = fetchUserAsync(userId)
    .thenApply(user -> user.getEmail()); // User → String (sync)
```

Think of it like `Stream.map()`. The function runs synchronously after the previous stage completes.

```
// Chain multiple thenApply for sequential transformations
fetchUserAsync(userId)
    .thenApply(user -> user.getUsername()) // User → String
    .thenApply(name -> name.toUpperCase()) // String → String
    .thenApply(name -> "Hello, " + name);  // String → String
```

thenCompose — Asynchronous chaining

Use when your transformation **returns another CompletableFuture**:

```
CompletableFuture<List<Order>> ordersFuture = fetchUserAsync(userId)
    .thenCompose(user -> fetchOrdersAsync(user.getId())); // User → CF<List<Ord
```

Think of it like `Stream.flatMap()`. It "flattens" the nested future.

The Classic Mistake: Nested Futures

Here's the bug I've seen in countless code reviews:

```
// ❌ WRONG: Using thenApply when you need thenCompose
CompletableFuture<CompletableFuture<List<Order>>> nested = fetchUserAsync(userId)
    .thenApply(user -> fetchOrdersAsync(user.getId()));
// Result: CompletableFuture<CompletableFuture<List<Order>>> 🤢
```

You've wrapped a future inside another future. Now you need to unwrap twice:

```
// Ugly double unwrap
List<Order> orders = nested.join().join(); // Don't do this!
```

The fix: Use `thenCompose`

```
// ✅ CORRECT: thenCompose flattens automatically
CompletableFuture<List<Order>> flat = fetchUserAsync(userId)
    .thenCompose(user -> fetchOrdersAsync(user.getId()));
// Result: CompletableFuture<List<Order>> ✓
```

The rule

If your function returns...	Use...
T (regular value)	thenApply
CompletableFuture<T>	thenCompose

Real-World Example: Profile Enrichment

Let's build something real. We need to:

1. Fetch user
2. Fetch their preferences
3. Fetch their loyalty points
4. Combine into a `UserProfile`

The naive sequential approach

```
public CompletableFuture<UserProfile> enrichProfile(Long userId) {
    return fetchUser(userId)
        .thenCompose(user ->
```

```

        fetchPreferences(user.getId())
            .thenCompose(prefs ->
                fetchLoyaltyPoints(user.getId())
                    .thenApply(points ->
                        new UserProfile(user, prefs, points)
                    )
            )
    );
}

```

This works, but it's **sequential**. Each call waits for the previous one, even though `preferences` and `loyaltyPoints` don't depend on each other.

The parallel approach

```

public CompletableFuture<UserProfile> enrichProfileParallel(Long userId) {
    return fetchUser(userId)
        .thenCompose(user -> {
            // Launch BOTH in parallel - they don't depend on each other
            CompletableFuture<List<String>> prefsFuture = fetchPreferences(user);
            CompletableFuture<Integer> pointsFuture = fetchLoyaltyPoints(user);

            // Combine when BOTH complete
            return prefsFuture.thenCombine(pointsFuture, (prefs, points) ->
                new UserProfile(user, prefs, points)
            );
        });
}

```

Same result, but `preferences` and `loyaltyPoints` fetch concurrently.

Parallel Execution: `allOf`, `anyOf`, `thenCombine`

When you have multiple independent operations, run them in parallel.

thenCombine — Combine exactly two futures

```
CompletableFuture<User> userFuture = fetchUser(userId);
CompletableFuture<List<Order>> ordersFuture = fetchOrders(userId);

// Both run in parallel, combine when done
CompletableFuture<String> summary = userFuture.thenCombine(ordersFuture,
    (user, orders) -> user.getName() + " has " + orders.size() + " orders"
)
```

allOf — Wait for all (three or more futures)

```
CompletableFuture<User> userFuture = fetchUser(userId);
CompletableFuture<List<Order>> ordersFuture = fetchOrders(userId);
CompletableFuture<List<Payment>> paymentsFuture = fetchPayments(userId);

// Launch all three in parallel
CompletableFuture<DashboardData> dashboard =
    CompletableFuture.allOf(userFuture, ordersFuture, paymentsFuture)
        .thenApply(ignored -> {
            // All futures are complete - join() won't block
            return new DashboardData(
                userFuture.join(),
                ordersFuture.join(),
                paymentsFuture.join()
            );
        });
```

Important: `allOf` returns `CompletableFuture<Void>`. You need to extract results from the original futures.

anyOf — First result wins (racing)

```
CompletableFuture<User> primaryService = fetchFromPrimary(userId);
CompletableFuture<User> backupService = fetchFromBackup(userId);

// First response wins
CompletableFuture<User> fastest = CompletableFuture.anyOf(primaryService, backupService)
    .thenApply(result -> (User) result); // anyOf returns Object
```

Use cases:

- Redundant requests (hedge your bets)
- Timeouts (race against a delayed future)
- Cache-aside (race cache vs database)

Why NOT ForkJoinPool.commonPool()

By default, `CompletableFuture.supplyAsync()` uses `ForkJoinPool.commonPool()`. This seems convenient, but it's **dangerous in production**.

The problems

1. **Shared across the entire JVM.** Every library using `CompletableFuture` shares this pool. One misbehaving dependency can starve your application.
2. **Sized for CPU-bound work.** Pool size = CPU cores - 1. For I/O-bound work (database calls, HTTP requests), this is way too small.

3. **No visibility.** Threads are named `ForkJoinPool.commonPool-worker-N`. Good luck debugging which operation is stuck.

The solution: Custom executors

```
// Create a dedicated executor for I/O operations
ExecutorService ioExecutor = Executors.newFixedThreadPool(
    Runtime.getRuntime().availableProcessors() * 2,
    new ThreadFactory() {
        private final AtomicInteger counter = new AtomicInteger(0);

        @Override
        public Thread newThread(Runnable r) {
            Thread t = new Thread(r, "io-pool-" + counter.incrementAndGet());
            t.setDaemon(true);
            return t;
        }
    }
);

// Use it explicitly
CompletableFuture.supplyAsync(() -> database.query(), ioExecutor);
```

Thread pool sizing guidelines

Workload	Formula	Example (8 cores)
I/O-bound	$\text{CPU} \times 2\text{-}4$	16-32 threads
CPU-bound	CPU	8 threads
Mixed	Separate pools	I/O: 16, CPU: 8

For I/O-bound work, threads spend most of their time waiting. More threads = more concurrent operations.

Virtual Threads: The Game Changer (Java 21+)

Java 21 introduced virtual threads, which fundamentally change the equation:

```
Executor virtualExecutor = Executors.newVirtualThreadPerTaskExecutor();  
CompletableFuture.supplyAsync(() -> database.query(), virtualExecutor);
```

Why virtual threads are special

- **Lightweight:** Millions of virtual threads are practical
- **No pool sizing:** Create as many as you need
- **Blocking is OK:** When a virtual thread blocks on I/O, the carrier thread is freed

```
// This is FINE with virtual threads  
CompletableFuture.supplyAsync(() -> {  
    User user = database.query(); // Blocks, but doesn't waste OS thread  
    return user;  
}, virtualExecutor);
```

When to use virtual threads

- ✓ I/O-bound operations (database, HTTP, file I/O)
- ✓ High concurrency scenarios

✗ CPU-bound operations (still use platform threads)

Sequential vs Parallel: A Performance Comparison

Let's see the real difference:

```
// ✗ SEQUENTIAL: Each operation waits for the previous
public CompletableFuture<Dashboard> loadSequential(Long userId) {
    return fetchUser(userId) // 100ms
        .thenCompose(user ->
            fetchOrders(userId) // +150ms
                .thenCompose(orders ->
                    fetchPayments(userId) // +120ms
                        .thenApply(payments ->
                            new Dashboard(user, orders, payments)))));
}
// Total: 370ms

// ✔ PARALLEL: Independent operations run concurrently
public CompletableFuture<Dashboard> loadParallel(Long userId) {
    CompletableFuture<User> userF = fetchUser(userId); // 100ms
    CompletableFuture<List<Order>> ordersF = fetchOrders(userId); // 150ms (parallel)
    CompletableFuture<List<Payment>> paymentsF = fetchPayments(userId); // 120ms
    return CompletableFuture.allOf(userF, ordersF, paymentsF)
        .thenApply(v -> new Dashboard(userF.join(), ordersF.join(), paymentsF.join()));
}
// Total: max(100, 150, 120) = 150ms
```

2.5x faster with the same operations.

Quick Reference: Composition Methods

Method	Use Case	Example
<code>thenApply(fn)</code>	Sync transformation	<code>.thenApply(user -> user.email())</code>
<code>thenCompose(fn)</code>	Async chaining	<code>.thenCompose(user -> fetchOrders(user.id()))</code>
<code>thenCombine(cf, fn)</code>	Combine two futures	<code>cf1.thenCombine(cf2, (a, b) -> ...)</code>
<code>allOf(cf...)</code>	Wait for all	<code>allOf(cf1, cf2, cf3).thenApply(...)</code>
<code>anyOf(cf...)</code>	First wins	<code>anyOf(cf1, cf2).thenApply(...)</code>

Key Takeaways

1. **`thenApply`** for sync transformations, **`thenCompose`** for async chaining. This is the most important rule.
2. **Run independent operations in parallel.** Use `allOf` or `thenCombine` to combine results.
3. **Never use `ForkJoinPool.commonPool()` in production.** Create dedicated executors with meaningful thread names.
4. **Size your pools for your workload.** I/O-bound: 2–4× CPU cores. CPU-bound: CPU cores.
5. **Consider virtual threads (Java 21+).** They eliminate most pool sizing headaches for I/O-bound work.

Coming Up: Part 3 — Error Handling & Spring Boot Integration

Async code can fail in surprising ways. Exceptions don't bubble up like you expect. In Part 3, we'll cover:

- The error handling trio: `exceptionally`, `handle`, `whenComplete`
- Handling partial failures in parallel operations
- Spring Boot executor configuration
- Why `@Async` might be hurting more than helping

See you there.

Check out the GitHub repo for <https://github.com/mo0rti/Learning-Completable-Future>

[*← Part 1: The Foundation*](#) | [*Part 3: Spring Boot Integration*](#) →

Spring Boot

Java

Concurrency

Asynchronous Programming

Completablefuture



Written by Morty

648 followers · 40 following

Follow

A Software Engineer busy creating top-notch apps. <https://mortitech.com/>

No responses yet

